

Add ProteinMPNN as a Tool for the ImmunoVerse Agent

Student intern assignment — build a Python module that lets the AI agent design & score protein sequences from a 3D backbone.

Intern: Eric Mentors: Aman Sharma & Xinya Liu — Yarmarkovich Lab Effort: ~2–3 weeks part-time

Deliverable in one line: a single Python file, `proteinmpnn_tools.py`, that wraps ProteinMPNN so the ImmunoVerse agent can **design** and **score** protein sequences for a given 3D backbone structure.

0. The big picture (read first)

The **ImmunoVerse agent** is an AI assistant (an LLM) that calls **tools** — ordinary Python functions — to do real scientific work. It already has a tool `analyze_pdb_interactions(...)` that scores binding contacts in a PDB structure. When a user asks "score the binding in this structure," the LLM calls that function and explains the JSON it gets back.

Your job: add a new capability — **ProteinMPNN**, a deep-learning model that, given a protein **backbone** (the 3D shape), proposes amino-acid **sequences** that should fit that shape, and can also **score** existing sequences against a backbone. Read the capability summary in

`ProteinMPNN/ProteinMPNN_Capabilities_Summary.pdf` for background.

Scope: You write **only** the function module `proteinmpnn_tools.py` that runs ProteinMPNN and returns clean structured results. You are **not** wiring it into the live agent or deploying it — your mentor does that. But write your functions in the **exact same style** as the existing tools so wiring them in is trivial.

1. Learning objectives

1. Run a published deep-learning research tool (ProteinMPNN) from the command line.
2. Wrap a command-line program as a clean, documented Python function.
3. Parse a scientific output format (FASTA with metadata headers) into structured data.
4. Match an existing codebase's conventions (return shapes, error handling, file lookup).
5. Write a small test that proves your function works on a real structure.

2. Required reading

- **Paper:** Dauparas et al., *Robust deep learning-based protein sequence design using ProteinMPNN*, Science 2022. DOI: 10.1126/science.add2187 — <https://www.science.org/doi/10.1126/science.add2187>
- **Official code + examples:** <https://github.com/dauparas/ProteinMPNN> — read `README.md` and the `examples/` folder. **The example scripts are your spec** for which flags do what.
- **Capability summary (in repo):** `ProteinMPNN/ProteinMPNN_Capabilities_Summary.pdf`

- **The convention you must copy:** `aibinder_tools.py` in the repo root. Read it end-to-end before writing code. It is your template.

3. Part A — Understand ProteinMPNN by RUNNING it (before any coding)

Never wrap a tool you haven't run by hand. Do these manual runs first and save the outputs; you'll reproduce them from Python later.

A.1 Set up the environment

```
git clone https://github.com/dauparas/ProteinMPNN.git
conda create -n proteinmpnn python=3.10 -y
conda activate proteinmpnn
pip install torch numpy          # CPU torch is fine for testing
```

ProteinMPNN ships its own model weights inside the repo (`vanilla_model_weights/`) — no separate download. Keep the cloned repo **separate** from our repo; do not commit it.

A.2 Manual run #1 — design sequences for one structure

We'll use a **published** RCSB structure: **8EK5** — the engineered scFv "10LH" bound to the PHOX2B peptide on HLA-A*24:02 (a neuroblastoma target). Its chains are **A** = HLA heavy chain, **B** = β 2-microglobulin, **C** = PHOX2B peptide, **E** = the scFv binder. Download it:

```
wget https://files.rcsb.org/download/8EK5.pdb
# or: curl -O https://files.rcsb.org/download/8EK5.pdb
```

Now redesign the **scFv binder (chain E)** while keeping the pHLA target (chains A, B, C) as fixed context — exactly the real ImmunoVerse use case:

```
python protein_mpnn_run.py \
  --pdb_path 8EK5.pdb \
  --pdb_path_chains "E" \
  --out_folder ./mpnn_out \
  --num_seq_per_target 4 \
  --sampling_temp "0.1" \
  --seed 37 --batch_size 1
```

Open `./mpnn_out/seqs/8EK5.fa` and study the format (Appendix C). Identify the native sequence line, the sampled designs, and the `score`, `global_score`, `seq_recovery` values.

A.3 Manual run #2 — score an existing sequence (score-only)

Reproduce `examples/example_3_score_only.sh` (and the `..._from_fasta` variant) on the same PDB. Note where the score is reported (stdout and/or an `.npz`).

A.4 Notes

In `ProteinMPNN/ERIC_notes.md`, record the exact commands, what each flag did, what the outputs looked like, and any confusion. This is part of the deliverable.

Milestone 1 checkpoint: show your mentor the two runs working + your notes before writing the wrapper.

4. Part B — Study our conventions (copy these exactly)

Open `aibinder_tools.py`. Your file will look just like it:

1. **Lazy imports + a `_check_deps()` function.** Import heavy libs (torch) and ProteinMPNN inside `try/except`, set a flag, and have `_check_deps()` return an error dict if something's missing — never crash on import. (lines ~17–73)
2. **Every function returns a dict:** `{"success": True, ...}` or `{"success": False, "error": "<message>"}`. **Never raise** out of a public function — catch and return the error dict.
3. **Resolve PDBs with the shared helper.** Reuse `_find_pdb(pdb_path)` (lines ~76–121) so a user can pass just a filename. Simplest: `from aibinder_tools import _find_pdb`.
4. **All return values must be JSON-serializable.** Convert numpy scalars to native Python (`float(x) / int(x)`), or the `_native()` pattern, line ~744). A stray `numpy.float32` breaks the agent.
5. **Paths from env vars with defaults** (lines ~48–56). Use the same `BASE_DIR` idea.

5. Part C — Build `proteinmpnn_tools.py`

Create the file in the **repo root** (next to `aibinder_tools.py`). Exact signatures and return shapes are in **Appendix A** — follow them precisely.

Milestone 2 REQUIRED — `proteinmpnn_design_sequences(...)`

Given a PDB path + which chain(s) to design, run ProteinMPNN and return the generated sequences with scores.

1. `_check_deps()` — bail early with an error dict if torch / ProteinMPNN missing.
2. `_find_pdb(pdb_path)` — resolve the file; error dict if not found.
3. Build a **unique temp output folder** with `tempfile.mkdtemp()` (concurrent calls must not clobber each other).
4. Call ProteinMPNN with `subprocess.run(..., capture_output=True, text=True, timeout=...)` — do **not** import its internals. Locate `protein_mpnn_run.py` via env var `PROTEINMPNN_DIR` (Appendix C).
5. Check `returncode`; if non-zero, return an error dict including trimmed `stderr`.
6. **Parse** `seqs/<stem>.fa` (Appendix C). Return native sequence + score, and a list of designs each with `sequence`, `score`, `global_score`, `seq_recovery`, `sample`, `temperature`.
7. Clean up the temp folder (keep the `.fa` path in the result for debugging, like Albinder keeps the `.pml` path).

Milestone 3 REQUIRED — `proteinmpnn_score_sequence(...)`

Given a PDB backbone + an amino-acid sequence (or FASTA), run ProteinMPNN **score-only** and return the score(s). Lower = more compatible. Read the score from stdout or the `.npz` — pick the simpler reliable option and document your choice.

Milestone 4 STRETCH — only if time allows

Pick **one**:

- **Fixed positions:** let the caller protect certain residues using ProteinMPNN's `helper_scripts/` to build the `fixed_positions_jsonl` (`examples/example_4.sh`).

- **Probability profile:** return per-position amino-acid probabilities (`--unconditional_probs_only` , `examples/example_7.sh`) as a table.

Do not start the stretch goal until Milestones 2 and 3 are reviewed and accepted.

6. Testing (required)

1. Add an `if __name__ == "__main__":` block that runs `proteinmpnn_design_sequences` on the downloaded `8EK5.pdb` (designing chain `E`) and prints the result (smoke test via `python proteinmpnn_tools.py`).
2. Write a **pure-function unit test** for your FASTA parser (no torch). Put a real sample `.fa` text in the test, feed it to your parser, assert scores/sequences are correct. File: `tests/test_proteinmpnn_parse.py` . The parser must be a separate function (e.g. `_parse_mpnn_fasta(text) -> list[dict]`) so it's testable without the model.

7. Definition of Done

- ☐ `proteinmpnn_design_sequences(...)` returns the documented dict on `8EK5.pdb` (designing chain `E`), with ≥ 1 designed sequence and numeric scores.
- ☐ `proteinmpnn_score_sequence(...)` returns a numeric score for a given sequence.
- ☐ Missing-dependency & bad-input cases return `{"success": False, "error": ...}` — never crash.
- ☐ All returned values are JSON-serializable (verify with `json.dumps(result)`).
- ☐ Code style matches `aibinder_tools.py` (lazy imports, `_check_deps` , `_find_pdb` , docstrings).
- ☐ FASTA-parser unit test passes (`pytest tests/test_proteinmpnn_parse.py`).
- ☐ `ERIC_notes.md` documents your manual runs.
- ☐ No model weights, large files, or cloned ProteinMPNN repo committed (check `git status`).

8. How to submit

1. Branch: `git checkout -b eric/proteinmpnn-tool`
2. Commit `proteinmpnn_tools.py` , `tests/test_proteinmpnn_parse.py` , `ProteinMPNN/ERIC_notes.md` .
3. Open a PR titled **"Add ProteinMPNN tool module"** , tag your mentor, paste an example result dict + test output.
4. Do **not** edit `api_server.py` , `tool_specs.py` , or any agent-wiring file — the mentor handles registration.

9. Gotchas & tips

- **Run by subprocess, not by importing ProteinMPNN internals** — far more robust.
- `--sampling_temp` is a **string** (e.g. `"0.1"`). Keep one temperature per call.
- **Score sign:** the score is a negative log-likelihood — **lower is better**. Say so in your docstring.
- **Temp folders:** fresh `tempfile.mkdtemp()` per call.
- **Timeouts:** give `subprocess.run` a `timeout` (e.g. 300s) and return a clean error if exceeded — the agent must not hang.

- **Don't hardcode paths.** ProteinMPNN via `PROTEINMPNN_DIR` ; PDBs via `_find_pdb` .
- **CPU mode is fine for now.** GPU is your mentor's deployment concern.

Appendix A — Exact function contracts (follow precisely)

```
def proteinmpnn_design_sequences(
    pdb_path: str,
    chains_to_design: str = "",          # e.g. "A" or "A B"; "" = design all chains
    num_sequences: int = 8,              # how many candidate sequences to generate
    sampling_temp: float = 0.1,         # 0.1 conservative ... 0.3 diverse
    seed: int = 37,
    model_name: str = "v_48_020",
) -> dict:
    """
    {
        "success": True,
        "pdb_file": "<resolved path>",
        "designed_chains": ["A"],
        "fixed_chains": ["B"],
        "model_name": "v_48_020",
        "sampling_temp": 0.1,
        "num_sequences": 8,
        "native_sequence": "MKT...",
        "native_score": 1.17,            # lower = better
        "sequences": [
            {"sample": 1, "sequence": "NLN...", "score": 0.89,
             "global_score": 1.03, "seq_recovery": 0.57, "temperature": 0.1},
            ...
        ],
        "out_fasta_path": "<temp>/seqs/<stem>.fa"
    }
    # or {"success": False, "error": "..."}
    """

def proteinmpnn_score_sequence(
    pdb_path: str,
    sequence: str = "",                 # one sequence to score; OR use fasta_path
    fasta_path: str = "",               # FASTA of sequences to score (optional)
    chains_to_score: str = "",          # "" = all chains
    seed: int = 37,
) -> dict:
    """
    {
        "success": True,
        "pdb_file": "<resolved path>",
        "scored": [
            {"name": "seq1", "score": 0.91, "global_score": 1.02},
            ...
        ],
        "note": "score = negative log-likelihood; lower means more compatible."
    }
    # or {"success": False, "error": "..."}
    """
```

Standard error shapes (copy from `aibinder_tools.py`):

```
{"success": False, "error": "Missing dependencies: torch, ProteinMPNN. Install with: ..."}
{"success": False, "error": "PDB file not found: <path>"}
{"success": False, "error": "ProteinMPNN failed: <trimmed stderr>"}
```

Appendix B — Registration for the mentor, not Eric

Once accepted, add `@register_tool(...)` above each function (like `aibinder_tools.py`), then import `proteinmpnn_tools` where `aibinder_tools` is imported so `TOOL_REGISTRY` picks them up.

```
from tool_specs import register_tool

@register_tool(
    name="proteinmpnn_design_sequences",
    description=(
        "Design amino-acid sequences for a protein BACKBONE using ProteinMPNN. "
        "Given a PDB structure, generate candidate sequences predicted to fold to "
        "that shape, ranked by score (lower = more compatible).",
    ),
    parameters={
        "type": "object",
        "properties": {
            "pdb_path": {"type": "string", "description": "Path/filename of the PDB."},
            "chains_to_design": {"type": "string", "default": "",
                                "description": "Chain(s) to redesign, e.g. 'A' or 'A B'. Empty = all."},
            "num_sequences": {"type": "integer", "default": 8},
            "sampling_temp": {"type": "number", "default": 0.1,
                              "description": "0.1 conservative ... 0.3 diverse."},
        },
        "required": ["pdb_path"],
    },
)
def proteinmpnn_design_sequences(...):
    ...
```

Deployment note: add `torch` to `deploy/requirements.txt`, bundle/clone ProteinMPNN into the image, set `PROTEINMPNN_DIR`. (`biopython` is already a dependency.)

Appendix C — ProteinMPNN reference

Locating the code (no hardcoding):

```
import os
PROTEINMPNN_DIR = os.environ.get("PROTEINMPNN_DIR", "/opt/ProteinMPNN")
RUN_SCRIPT = os.path.join(PROTEINMPNN_DIR, "protein_mpnn_run.py")
```

Design command (single PDB, example_3 style):

```
python protein_mpnn_run.py \
  --pdb_path STRUCT.pdb --pdb_path_chains "A" \
  --out_folder OUT --num_seq_per_target 8 \
  --sampling_temp "0.1" --seed 37 --batch_size 1
```

Output sequences land in `OUT/seqs/<pdb_stem>.fa`.

Output FASTA format to parse (`_parse_mpnn_fasta`):

```
>8EK5, score=1.0241, global_score=1.1180, fixed_chains=['A', 'B', 'C'], designed_chains=['E'],
model_name=v_48_020, seed=37
EVQLVESGGGLVQPGGSLRLSCAAS...          <- NATIVE scFv (first record)
>T=0.1, sample=1, score=0.8123, global_score=0.9450, seq_recovery=0.6010
EVQLQESGPGLVKPSSETLSLTCTVS...          <- designed sample 1
>T=0.1, sample=2, score=0.8367, global_score=0.9588, seq_recovery=0.5874
...
```

Parsing rules:

- Records are `>header` then sequence line(s).
- The **first** record is the native/input sequence; its header carries `fixed_chains`, `designed_chains`, `model_name`, `seed`, and the native `score`.
- Each later header is comma-separated `key=value: T, sample, score, global_score, seq_recovery`. Split on commas, then on the first `=`.
- `score` / `global_score` are floats (negative log-likelihood; lower is better).

Score-only (example_3_score_only / `_from_fasta`): add `--score_only 1` and, to score external sequences, `-path_to_fasta YOUR.fa`. Inspect stdout and the output folder to decide where to read the score; document your choice.

Helper scripts (stretch goal) in `helper_scripts/`: `parse_multiple_chains.py`, `make_fixed_positions_dict.py`, `assign_fixed_chains.py`, `make_tied_positions_dict.py`. The `examples/*.sh` scripts show how to chain them.